SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

Multithreading Practice - CO3

← Back

QUESTIONS

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Q1 Multiple Threads Using a Single Class 10 marks

Topic: Thread class, multiple threads, if-else

Question:
Write a Java program using a single thread class to create three threads named Prime, Armstrong, and Reverse.

The program should contain a common run() method. Based on the thread name, use if-else statements to perform the following operations on a given number:

Prime → Check whether the number is prime.
Armstrong → Check whether the number is an Armstrong number.
Reverse → Find the reverse of the number.

Create and start all three threads from the main() method.

Input: 153

Expected Output:

Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351

Your Code

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args)
        Scanner scanner = new Scanner(Syst
        if (scanner.hasNextInt()) {
            int number = scanner.nextInt()
            NumberOperations primeThread =
            NumberOperations armstrongThre
            NumberOperations reverseThreac
            try {
                primeThread.start();
                primeThread.join();
                armstrongThread.start();
                armstrongThread.join();
                reverseThread.start();
                reverseThread.join();
            } catch (InterruptedException
                e.printStackTrace();
            }
        }
        scanner.close();
    }
}

class NumberOperations extends Thread {
    private int num;

    public NumberOperations(String name, 
        super(name);
        this.num = num;
    }

    @Override
    public void run() {
        String name = getName();

        if (name.equalsIgnoreCase("Prime")
            boolean isPrime = true;
            if (num <= 1) {
                isPrime = false;
            } else {
                for (int i = 2; i <= num 
                    if (num % i == 0) {
                        isPrime = false;
                        break;
                    }
                }
            }
```

```
            if (isPrime) {
                System.out.println("Prime
            } else {
                System.out.println("Prime
            }
        } else if (name.equalsIgnoreCase('
            int original = num;
            int temp = num;
            int digits = 0;
            while (temp > 0) {
                digits++;
                temp /= 10;
            }
            temp = num;
            int sum = 0;
            while (temp > 0) {
                int rem = temp % 10;
                sum += Math.pow(rem, digit
                temp /= 10;
            }
            if (sum == original) {
                System.out.println("Armstr
            } else {
                System.out.println("Armstr
            }
        } else if (name.equalsIgnoreCase('
            int temp = num;
            int rev = 0;
            while (temp > 0) {
                int rem = temp % 10;
                rev = rev * 10 + rem;
                temp /= 10;
            }
            System.out.println("Reverse Th
        }
    }
}
```

Input

153

Output

```
Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351
```

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks

Submitted

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

Multithreading Practice - CO3

Back

QUESTIONS

Q1 ✓
Multiple Threads Using a Single Class
10 marks

Q2 ✓
Multiple thread classes
20 marks

Q3 ✓
Thread Synchronization
10 marks

3/3 answered

Q2 Multiple thread classes

20 marks

Question:
Write a Java program to create three different thread classes to perform different operations on the same number.

Create the following classes:

PrimeThread → Check whether the given number is prime.
FactorialThread → Find the factorial of the given number.
ReverseThread → Find the reverse of the given number.

Each class should extend the Thread class and override the run() method.

Create and start all three threads from the main() method.

Input: 7

Expected Output:

Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

Note: Since the threads execute concurrently, the order of output may vary.

Your Code

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args)
        Scanner scanner = new Scanner(Syst
        if (scanner.hasNextInt()) {
            int number = scanner.nextInt()
            PrimeThread primeThread = new
            FactorialThread factorialThrea
            ReverseThread reverseThread =
            try {
                primeThread.start();
                primeThread.join();
                factorialThread.start();
                factorialThread.join();
                reverseThread.start();
                reverseThread.join();
            } catch (InterruptedException
                e.printStackTrace();
            }
        }
        scanner.close();
    }
}
class PrimeThread extends Thread {
    private int num;
    public PrimeThread(int num) {
        this.num = num;
    }
    public void run() {
        boolean isPrime = true;
        if (num <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i <= num / 2;
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        if (isPrime) {
            System.out.println("Prime: " -
        } else {
            System.out.println("Prime: " -
        }
    }
```

```
47 }
48 class FactorialThread extends Thread {
49     private int num;
50     public FactorialThread(int num) {
51         this.num = num;
52     }
53     public void run() {
54         long fact = 1;
55         for (int i = 1; i <= num; i++) {
56             fact *= i;
57         }
58         System.out.println("Factorial: " -
59     }
60 }
61 class ReverseThread extends Thread {
62     private int num;
63     public ReverseThread(int num) {
64         this.num = num;
65     }
66     public void run() {
67         int temp = num;
68         int rev = 0;
69         while (temp > 0) {
70             int rem = temp % 10;
71             rev = rev * 10 + rem;
72             temp /= 10;
73         }
74         System.out.println("Reverse: Reve
75     }
76 }
```

Input

```
7
```

Output

```
Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7
```

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

20.00 / 20 marks

Submitted

SIMATS ENGINEERING

SIMATS Online Programming Lab

JAVA PROGRAMMING

MD Imthiaz I
192521360RD

Multithreading Practice - CO3

Back

QUESTIONS

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Q3 Thread Synchronization

10 marks

opic: synchronized, shared resource, race condition

Question:
Write a Java program to demonstrate thread synchronization using a shared Counter object.

Create a class Counter containing an integer variable count, initially set to 0.

Create two threads, ThreadA and ThreadB. Both threads should increment the same count variable 1000 times.

Implement the increment() method using the synchronized keyword so that only one thread can modify the counter at a time.

After both threads complete their execution, display the final value of count.

Expected Output:

ThreadA completed
ThreadB completed
Final Count = 2000

Requirements:

Use a shared Counter object.
Both threads must access the same Counter object.
Use synchronized for the increment operation.

Use join() to ensure both threads finish before displaying the final count.

Your Code

```
public class Main {
    public static void main(String[] args)
        counter co = new counter();
        ThreadA T1 = new ThreadA(co);
        ThreadB T2 = new ThreadB(co);
        try {
            T1.start();
            T1.join();
            T2.start();
            T2.join();
        } catch (InterruptedException e)
            e.printStackTrace();
        }
        System.out.println("Final Count =
    }
}
class counter {
    int count = 0;
    public synchronized void counterIncrer
        count++;
    }
}
class ThreadA extends Thread {
    counter c;
    ThreadA(counter co) {
        c = co;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            c.counterIncrement();
        }
        System.out.println("ThreadA comple
    }
}
class ThreadB extends Thread {
    counter c;
    ThreadB(counter co) {
        c = co;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            c.counterIncrement();
```

```
43          }
44          System.out.println("ThreadB comple
45      }
46  }
```

Input

```
stdin input
```

Output

```
ThreadA completed
ThreadB completed
Final Count = 2000
```

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks

Submitted